从零做一个自己的桌面宠物

> 适用对象：没有任何动画制作经验，也能跟着做完。
>
> 你会得到：一个装进 DSH App 的桌宠 —— 会呼吸、会回应点击、你干活时它跟着换姿势。
>
> 配套工具：`DSH桌宠工作台.html`（浏览器打开即用，不用装任何软件）
>
> ⚠️ 最简单的方式：你有视频就行。 AI 生成的、剪映导出的、网上下载的 mp4， 直接拖进工作台 —— 有背景也能自动抠掉，不用先拆帧、不用装 ffmpeg。

目录

- 第 0 章 五分钟做出第一个宠物
- 第 1 章 原理：宠物是怎么动起来的
- 第 2 章 准备素材
- 第 3 章 用工作台生成透明视频
- 第 4 章 配置动作与状态
- 第 5 章 装到手机上
- 第 6 章 命令行方式（进阶）
- 第 7 章 常见问题
- 附录 A 完整配置参考
- 附录 B ffmpeg 命令速查

第 0 章 五分钟做出第一个宠物

你只需要一个视频。 别的都不用管。

0.1 准备一个视频

任何来源都行：

来源	例子
AI 生视频	即梦、可灵、Runway 生成的 mp4
剪映 / CapCut	做好动画后导出 mp4
网上下载	找到的角色动画视频
自己录屏	游戏/动画里截一段

关键点：不管有没有背景都行。 白底、绿幕、纯色底，工作台会自动抠掉。

要求：

- 时长 1–5 秒（太长只取前 5 秒）
- 角色在画面里完整可见
- 背景尽量是纯色（纯白、纯绿最好）

0.2 打开工作台

双击 `DSH桌宠工作台.html`，用 Chrome 或 Edge 打开。

> ⚠️ 必须用 Chrome / Edge。Safari 和 Firefox 不支持透明视频录制。

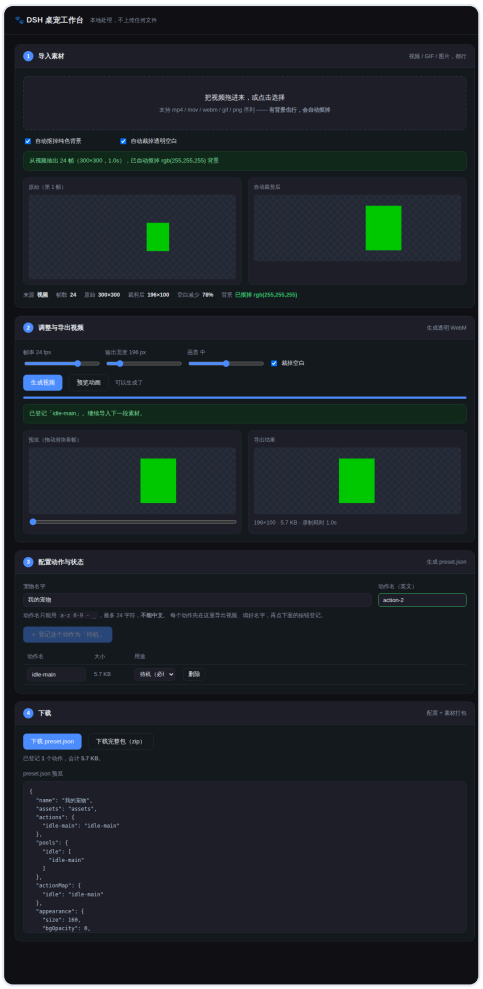
0.3 四步搞定

第一步：拖入视频

把 mp4 拖进虚线框。

工作台会自动完成：

- 逐帧提取（默认 24 帧/秒）
- 识别背景色并抠掉（白底 → 透明）
- 算出角色的实际范围并裁掉空白
- 显示「原始 vs 裁剪后」的对比



底部会显示处理结果，例如：

```
来源：视频 · 帧数 24 · 原始 300×300
裁剪后 196×100 · 空白减少 78%
背景：已抠掉 rgb(255,255,255)
```

第二步：点「生成视频」

等进度条走完（几秒，取决于帧数）。

生成完会显示：`196×100 · 5.7 KB · 录制耗时 1.0s`

第三步：登记动作

1. 在「动作名」填 `idle-main`（只能用英文小写+数字+连字符）
2. 点「＋ 登记这个动作为「待机」」

第四步：下载

点「下载完整包（zip）」，得到 `my-pet.zip`，里面有：

```
preset.json
assets/idle-main.webm
```

0.4 装到手机

1. 解压 `my-pet.zip`
2. 把整个文件夹推到手机：`adb push my-pet /sdcard/Download/`
3. 在 App 里对 AI 说：

> 帮我把 /sdcard/Download/my-pet 复制到 $DSH_HOME/deskpet/ 下， 然后用 deskpet 工具应用 preset 指向那里的 preset.json

1. 按 HOME 键退到后台，看悬浮球

就这些。 下面是完整版，教你怎么做出会思考、会生气、会撒娇的宠物。

第 1 章 原理：宠物是怎么动起来的

1.1 宠物 = 一堆短视频 + 一张调度表

悬浮球本身只是一个窗口。宠物之所以「活」，是因为它不停地换视频。

```
你的角色
  ├─ 待机呼吸.webm      ┐
  ├─ 打哈欠.webm        │  闲着时随机抽一段播
  ├─ 玩魔方.webm        ┘
  ├─ 思考.webm          →  你在发消息时播
  ├─ 干活.webm          →  跑工具时播
  ├─ 出错.webm          →  上一轮失败时播
  ├─ 点击回应.webm      →  你点它时播
  └─ 走路.webm          →  它自己溜达时播
```

你不需要写代码，只需要：

1. 做几个短视频（每个动作一段）
2. 写一个 JSON 文件，告诉 App「哪个状态播哪段」

1.2 七个状态：宠物什么时候演什么

DSH 会实时把 AI 的运行状态告诉宠物。一共七种：

状态	什么时候	建议做什么动作
`idle`	没在干活，等你	呼吸、发呆、小动作
`thinking`	AI 正在生成回复	思考、冒泡、挠头
`working`	正在跑工具/命令	忙碌、打字、搬东西
`waiting`	等你确认/输入	张望、招手、看表
`error`	上一轮失败了	垂头、冒汗、哭
`result`	工具刚跑完，AI 在消化结果	翻看、点头、清点
`success`	这一轮顺利完成（瞬时）	欢呼、跳跃、比心



> 上排从左到右：`idle` `thinking` `working` `waiting`；下排：`error` `result` `success`。 实际动作由你的素材决定，这里只是同一角色的不同瞬间。

关键点：只有 `idle` 是必须的。其它状态没配，就会回落到 `idle` —— 不会出错，只是不够生动。

1.3 动作池：让待机不呆滞

如果 `idle` 只有一段视频循环播，看久了会很假。

所以有「动作池」：把多段待机动作放进 `idle` 池，App 会播完一段随机抽下一段。

```
"pools": {
  "idle": ["呼吸", "打哈欠", "玩魔方", "发呆"],   // ← 随机链
  "click": ["开心", "害羞", "生气"],               // ← 点一下随机抽一个
  "move": ["走路A", "走路B"]                       // ← 自己溜达时用
}
```

六个特殊池名，写错了不会报错但也不生效，务必照抄：

池名	作用
`idle`	随机待机链。这是让宠物不呆滞的关键
`click`	点一下的反应。⚠️ 配了它，单击变成摸宠物，面板要长按打开
`drag`	被拎起来时的姿势
`move`	自己走路时的姿势（需开 `move.enabled`）
`turn`	转身姿势。播完才真的翻转朝向
`work-<状态>`	状态专属池，比如 `work-thinking` 每次进这个状态重新抽

1.4 为什么必须是透明视频

宠物贴在屏幕上，背景必须透明，否则会出现一个方块挡住下面的内容。

透明视频的格式要求：

项目	要求
容器	WebM
编码	VP9
像素格式	`yuva420p`（带 alpha）
标记	`alpha_mode=1`

工作台会帮你自动处理这些。

第 2 章 准备素材

2.1 你需要什么样的图

核心要求：透明背景。

项目	建议
格式	PNG（带 alpha 通道）或 GIF
尺寸	长边 400–600 px
背景	完全透明，不能是白色/黑色
角色位置	每一帧都要对齐，别抖动

2.2 四条获取素材的路子

路子 A：直接有视频（最省事，推荐）

这是最短路径。 工作台能直接吃视频，并且自动抠掉纯色背景。

来源	怎么拿
AI 生视频	即梦、可灵、Runway、Pika —— 生成后下载 mp4
剪映 / CapCut	做好动画导出 mp4
现成动画	网上下载的角色视频
录屏	从游戏/动画里截一段

直接拖进工作台即可。不需要先抠图。

> ⚠️ 背景要纯色（纯白/纯绿最佳）。如果背景是复杂场景（房间、街道），自动抠图会失败，需要先自己处理。

路子 B：AI 生图 + 抠图（想要精确控制时）

1. 用 AI 画图工具生成角色，提示词加「白色背景」「全身」「正面」「Q版」

2. 抠图：bgsub.cn / remove.bg / Photoshop「选择主体」
3. 导出 PNG

> 其实不抠也行 —— 直接给白底 PNG，工作台会自动抠掉白色。

路子 C：手绘

用 Procreate / Clip Studio / Photoshop 画，导出 PNG 序列。 画的时候打开「透明背景」图层，或者画白底（工作台会抠）。

路子 D：现成 GIF

网上找的 GIF 直接拖进工作台。如果是花底，需要先处理。

2.3 帧数要多少

动作类型	建议帧数	说明
呼吸 / 待机	8–16 帧	循环播放，帧少也自然
点击回应	12–24 帧	一次性播放
走路	8–12 帧	循环
思考	12–20 帧	循环

帧率：建议 24 fps。低于 12 会卡顿，高于 30 意义不大。

2.4 命名规范（重要）

如果你有多张 PNG，按文件名数字排序：

```
frame_0.png
frame_1.png
frame_2.png
...
frame_10.png    ← 注意：会排在 frame_2 后面（工作台做了自然排序）
```

工作台能正确识别 `frame_2` < `frame_10`，不需要补零。

第 3 章 用工作台生成透明视频

3.1 打开工作台

双击 `DSH桌宠工作台.html`。

必须用 Chrome 或 Edge。 工作台会在启动时检测，不支持会直接提示。

3.2 导入素材

支持什么

格式	处理方式
mp4 / mov / webm / mkv	逐帧提取（默认 24 帧/秒）+ 自动抠背景
gif	拆帧 + 自动抠背景
png / jpg 序列	直接读帧 + 自动抠背景

怎么导入

方式一：拖拽 —— 把文件拖进虚线框。

方式二：点击选择 —— 点虚线框选文件：

- 视频 / GIF：选一个
- PNG 序列：按住 Ctrl（Windows）或 Cmd（Mac）多选

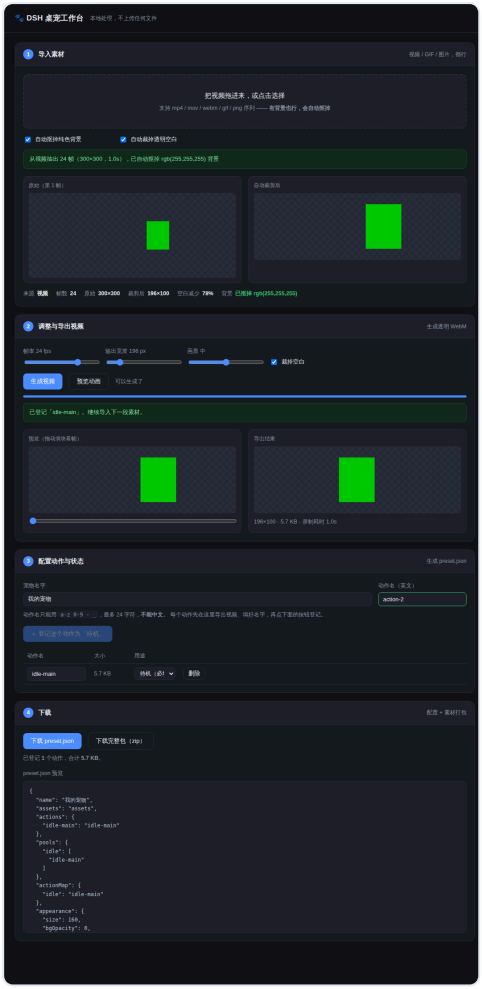
两个自动开关

导入区下面有两个勾选项，默认都开：

开关	作用
自动抠掉纯色背景	检测画面四角颜色，一致就把该颜色变透明（带羽化）
自动裁掉透明空白	算所有帧的并集包围盒，去掉四周空白

什么时候关掉自动抠背景：素材本来就是透明的（带 alpha 的 webm/png），或者背景是复杂场景不想被误抠。

导入后会看到什么



区域	说明
原始（第 1 帧）	你的原图
自动裁剪后	切掉透明空白后的样子
帧数 / 原始 / 裁剪后 / 空白减少	数据统计

「空白减少 82%」这个数字很重要 —— 它意味着碰撞箱（能点的范围）缩小了 82%，宠物会贴合得多。

3.3 调整参数

参数	作用	建议
帧率	每秒播几帧	24
输出宽度	视频的宽度（像素）	跟随裁剪后尺寸即可，或手动调
画质	低 / 中 / 高 / 最高	中（体积和质量平衡）
裁掉空白	是否自动裁剪	保持勾选

输出宽度怎么选：

- 太小 → 手机上放大后模糊
- 太大 → 体积超限，下发失败

公式：`输出宽度 ≈ 你希望的显示大小（dp）× 屏幕密度`

常见值：

- 160dp 显示 → 480px（3x 屏）
- 但 320px 通常也够看

3.4 预览

点「预览动画」看效果。或者拖「预览」下面的滑块逐帧检查。

3.5 生成视频

点「生成视频」，等进度条走完。

> ⏱ 生成需要时间：MediaRecorder 是按真实时间录制的， 所以 24 帧 @24fps 需要约 1 秒。48 帧就要 2 秒。

生成完显示：

```
120×104 · 1.4 KB · 录制耗时 0.3s
```

体积检查：

体积	判断
< 2 MB	完美
2–6 MB	可以，但注意总预算
> 6 MB	⚠️ 工作台会警告，需要降低画质或减小宽度

3.6 登记动作

1. 填动作名：只能用 `a-z 0-9 - _`，最多 24 字符，不能中文
2. 输入框会实时变色：绿色 = 合法，红色 = 非法
3. 点「+ 登记这个动作为…」
4. 在下拉框里选用途（待机 / 思考 / 干活 / 点击回应 / …）

登记后表格里会出现一行：

动作名	大小	用途
idle-main	1.4 KB	待机（必填）

重复做下一段素材：重新拖入新图片 → 生成 → 填新动作名 → 登记。

3.7 下载

按钮	得到什么
下载 preset.json	只有配置文件
下载完整包（zip）	preset.json + assets/ 下所有视频

推荐下载 zip。

第 4 章 配置动作与状态

4.1 preset.json 的结构

```
{
  "name": "我的宠物",           // 宠物名字
  "assets": "assets",          // 素材目录

  "actions": {                 // 动作名 → 文件名（不含扩展名）
    "idle-main": "idle-main",
    "think": "think"
  },

  "pools": {                   // 动作池
    "idle": ["idle-main"],
    "click": ["happy"]
  },

  "poolWeights": {             // 池的权重
    "idle": 10
  },

  "actionMap": {               // 七个状态 → 动作
    "idle": "idle-main",
    "thinking": "think"
  },

  "appearance": {              // 外观
    "size": 160,
    "bgOpacity": 0,
    "avatar": "none",
    "showDot": false
  }
}
```

4.2 关键字段详解

`appearance.size` —— 显示大小

范围 24–320，默认 160。

如果你的角色不是正方形，用 `boxWidth` / `boxHeight` 指定比例：

```
"appearance": {
  "size": 160,
  "boxWidth": 160,     // 长边 = size
  "boxHeight": 142     // 按比例自动算
}
```

> 原理：`boxWidth` / `boxHeight` 只表达长宽比，实际大小由 `size` 控制。 在 App 里拖「直径」滑块时，整体等比缩放。

`appearance.bgOpacity` —— 必须为 0

```
"bgOpacity": 0     // 背景全透明
```

如果不设 0，宠物周围会有一个半透明方块。

`workStatusTexts` —— 让宠物说话

头顶气泡的文字，按七种状态顺序给：

```
"workStatusTexts": [
  ["在发呆", "闲下来了"],         // idle
  ["正在想…", "让我理理思路"],    // thinking
  ["正在干活…"],                    // working
  ["等你确认"],                     // waiting
  ["出错了…"],                      // error
  ["搞定一步"],                     // result
  ["完成啦！"]                      // success
]
```

每档给多句，切换时随机抽一句。只配前几档也行。

`pools` 与 `poolWeights` —— 控制随机频率

```
"pools": {
  "idle": ["呼吸", "打哈欠", "玩魔方"]
},
"poolWeights": {
  "idle": 10    // 0 = 关掉随机链
}
```

`move` —— 自主走动

```
"move": {
  "enabled": true,      // 开启
  "intervalSec": 30,    // 每 30 秒走一次
  "minDp": 40,
  "maxDp": 120
}
```

4.3 ⚠️ 动作名规则（最容易踩的坑）

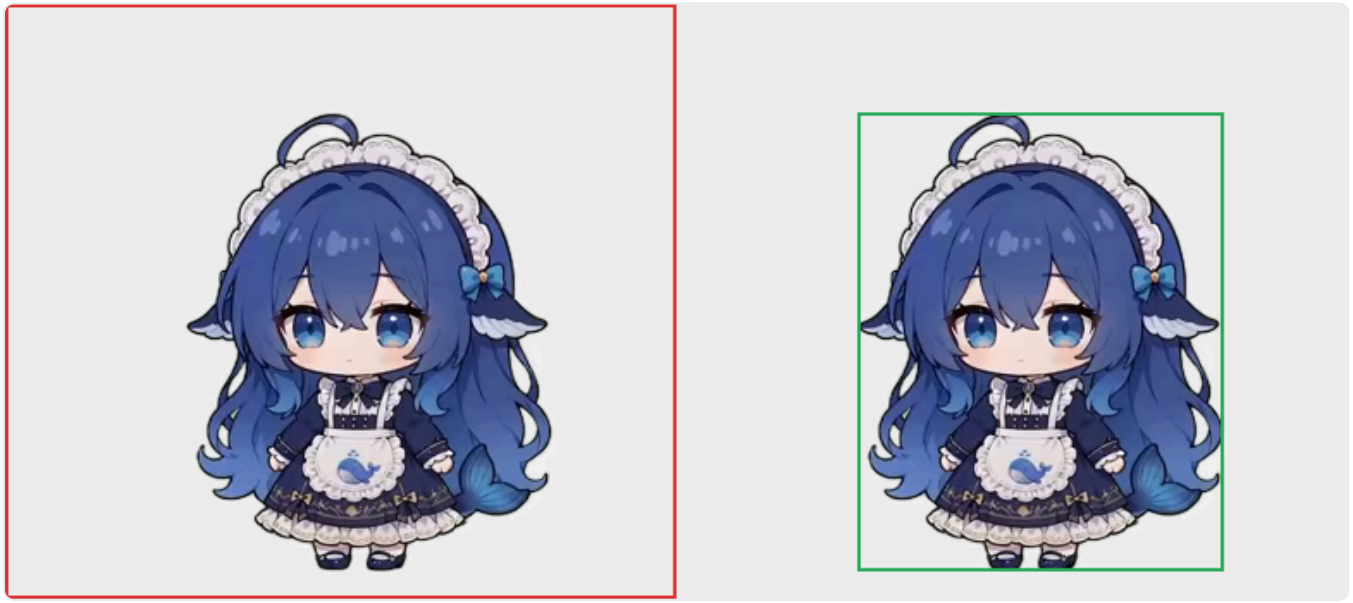
只能用 `a-z 0-9 - _`，最多 24 字符，不能中文。

```
"actions": {
  "呼吸": "idle"        // ❌ 错：中文动作名会被拒绝
  "breathe": "idle"     // ✅ 对
}
```

素材文件名用中文通常没问题，但动作名（`actions` 的 key）必须英文。

4.4 碰撞箱问题：为什么裁剪很重要

如果视频里角色周围有大片透明空白，整个画面都是可点区域 —— 用户点到空白处也会触发宠物，感觉很怪。



> 左：整个画面都是可点区域（红框），但角色只占中间。 右：裁剪到角色实际范围（绿框），碰撞箱贴合本体。

工作台的「自动裁剪」就是解决这个的。它算的是所有帧的并集：

- 如果每帧各自裁 → 尺寸不一致 → 切换动作时角色会跳
- 用并集 → 所有帧同一尺寸 → 平滑切换

第 5 章 装到手机上

5.1 方式一：让 AI 帮你装（推荐）

第一步：推到手机

```
adb push my-pet /sdcard/Download/
```

或者用微信/QQ 传文件到手机，再用文件管理器放到 Download。

第二步：让 AI 复制到私有目录

在 App 里对 AI 说：

> 帮我把 /sdcard/Download/my-pet 复制到 $DSH_HOME/deskpet/ 下

第三步：应用配置

> 用 deskpet 工具，preset 指向 /data/user/0/com.dsh.android/files/home/.dsh/deskpet/my-pet/preset.json

5.2 方式二：直接让 AI 改（最快，适合试水）

素材已经在手机上时，直接对 AI 说：

> 用 bubble 工具，把我的宠物设成： - idle 动作播 /sdcard/Download/idle.webm - 点击回应播 /sdcard/Download/happy.webm - 大小 180，背景透明

AI 会调用 `bubble` 工具下发。

5.3 让宠物在前台可见

悬浮球默认只在 App 退到后台时显示。想边调边看：

> 用 bubble 工具把 state 设成 collapsed

这样它就钉在前台了。

5.4 方式三：做成插件包（可分享）

如果你懂 npm，把目录打包成插件：

```
my-pet/
├── package.json
├── lib/index.js       ← 复制 dsh-app-pet 的
├── presets/
│   └── pet.json       ← 你的配置
└── assets/
    └── *.webm
```

`package.json`：

```
{
  "name": "dsh-app-pet-mine",
  "version": "1.0.0",
  "type": "module",
  "main": "./lib/index.js",
  "dsh": {
    "manifestVersion": 1,
    "bundle": { "patch": "./cordis.patch.yml" }
  }
}
```

然后 `npm publish`，用户：

```
dsh plugin --profile web add dsh-app-pet-mine
```

第 6 章 命令行方式（进阶）

不想用工作台？可以用 ffmpeg 手动处理。

6.1 安装 ffmpeg

Windows

```
winget install ffmpeg
```

macOS

```
brew install ffmpeg
```

Linux

```
sudo apt install ffmpeg
```

验证：

```
ffmpeg -version
```

6.2 转换命令

情况一：PNG 序列

```
ffmpeg -framerate 24 -i frame_%d.png \
  -c:v libvpx-vp9 -pix_fmt yuva420p -crf 30 -b:v 0 \
  -metadata:s:v:0 alpha_mode="1" \
  idle.webm
```

> `%d` 是数字占位符。如果文件是 `0001.png` 补零的，用 `%04d`。

情况二：GIF

```
ffmpeg -i your.gif \
  -c:v libvpx-vp9 -pix_fmt yuva420p -crf 30 -b:v 0 \
  -metadata:s:v:0 alpha_mode="1" \
  idle.webm
```

情况三：绿幕视频

```
ffmpeg -i green.mp4 \
  -vf "chromakey=0x00FF00:0.15:0.05,format=yuva420p" \
  -c:v libvpx-vp9 -pix_fmt yuva420p -crf 30 -b:v 0 \
  -metadata:s:v:0 alpha_mode="1" \
  idle.webm
```

6.3 参数说明

参数	作用	建议
`-framerate 24`	输入帧率	24
`-pix_fmt yuva420p`	必须，带 alpha	别改
`-crf 30`	质量，越小越好越大	28–34
`-b:v 0`	配合 crf 用恒定质量	别改
`-metadata:s:v:0 alpha_mode="1"`	必须，标记透明	别改

最常犯的错：漏了 `-pix_fmt yuva420p`，转出来不透明。

6.4 检查视频

```
ffprobe -v error -select_streams v:0 \
  -show_entries stream=pix_fmt -show_entries stream_tags=alpha_mode \
  -of default=nw=1 idle.webm
```

正确输出：

```
pix_fmt=yuv420p
TAG:ALPHA_MODE=1
```

> ⚠️ 看到 `pix_fmt=yuv420p` 别慌，这是正常的。 VP9 把 alpha 单独存在 `ALPHA_MODE` 里。关键是有没有 `TAG:ALPHA_MODE=1`。

6.5 裁剪空白

```
# <a id="先看第一帧-找出角色边界"></a>先看第一帧，找出角色边界
ffmpeg -c:v libvpx-vp9 -i idle.webm -vframes 1 frame.png

# <a id="然后裁剪-宽-高-x-y"></a>然后裁剪（宽:高:X:Y）
ffmpeg -c:v libvpx-vp9 -i idle.webm \
  -vf "crop=92:80:50:60" \
  -c:v libvpx-vp9 -pix_fmt yuva420p -crf 30 -b:v 0 \
  -metadata:s:v:0 alpha_mode="1" idle-cropped.webm
```

> 重要：所有动作要裁成同一个区域，否则切换时会跳。

第 7 章 常见问题

Q1：宠物显示成一个方块 / 有白色背景

原因：视频没有透明通道，或者 `bgOpacity` 不是 0。

排查：

```
ffprobe -v error -select_streams v:0 \
  -show_entries stream_tags=alpha_mode -of default=nw=1 idle.webm
```

没有 `TAG:ALPHA_MODE=1` → 重转，确认带了 `-pix_fmt yuva420p`。

Q2：碰撞箱（能点的范围）比角色大很多

原因：视频里角色周围有大片透明空白。

解决：用工作台的「自动裁剪」，或手动 `crop`。

Q3：切换动作时会闪一下

这是 App 端的问题，不是你的素材。旧版播放器换视频时会清空画面。

更新到最新版 APK 即可（已改为双缓冲，不再闪）。

Q4：动作名报错 / 不生效

检查：

- 只能 `a-z 0-9 - _`，不能中文、不能空格
- 最长 24 字符
- `actions` 里的 key 要和 `pools`、`actionMap` 里引用的一致

Q5：视频体积太大，下发失败

上限	值
单个动作	6 MB
一次下发的全部动作	12 MB

压缩方法（按效果排序）：

1. 调大 `-crf`：30 → 34（体积约减 30%）
2. 降分辨率：`-vf scale=320:-2`
3. 减帧数：`-r 12`
4. 缩短时长

Q6：宠物在屏幕上但完全不动

排查顺序：

1. `pools.idle` 是空的？→ 至少要有一个动作
2. `actionMap.idle` 指向的动作存在吗？→ 检查 `actions` 里有没有
3. 素材真的能播吗？→ 用播放器打开 webm 试试

Q7：点一下宠物，它没反应

原因：没配 `click` 池。

```
"pools": {
  "click": ["happy"]     // 加上这行
}
```

⚠️ 但注意：配了 **`click`** 池之后，单击变成摸宠物，打开面板要长按。

Q8：工作台打不开 / 提示不支持

必须用 Chrome 或 Edge。Safari 和 Firefox 不支持透明视频录制。

检查：地址栏输入 `chrome://version` 确认浏览器版本。

Q9：工作台生成的视频是黑底

原因：某些旧版 Chrome 的 MediaRecorder 不支持 alpha。

排查：把生成的 webm 拖进 Chrome 播放，如果显示黑底就是这个问题。

解决：升级 Chrome 到最新版，或改用 ffmpeg 命令行方式。

Q10：GIF 只导入了第一帧

原因：这个浏览器不支持 `ImageDecoder`（用来解 GIF 动画）。

解决：用 Chrome 最新版，或者先把 GIF 拆成 PNG 序列（用 ezgif.com/split）。

附录 A 完整配置参考

A.1 所有字段

```
{
  // ── 基本信息 ──────────────────────────
  "name": "宠物名",
  "description": "描述",
  "assets": "assets",              // 素材目录（相对 preset.json）

  // ── 动作定义 ──────────────────────────
  "actions": {
    "动作名": "文件名不含扩展名"       // 动作名必须 a-z0-9-_，≤24 字符
  },
  "sources": {
    "动作名": "中文说明"               // 仅供 list 命令显示
  },

  // ── 调度规则 ──────────────────────────
  "pools": {
    "idle":  ["动作1", "动作2"],     // 随机待机链
    "click": ["动作3"],               // 点击回应
    "drag":  ["动作4"],               // 拖拽姿势
    "move":  ["动作5"],               // 走路
    "turn":  ["动作6"],               // 转身
    "work-thinking": ["动作7"]        // 状态专属池
  },
  "poolWeights": {
    "idle": 10,                       // 0 = 关闭随机链
    "turn": 2
  },
  "actionMap": {
    "idle": "动作1",                  // 七个状态 → 动作
    "thinking": "动作7",
    "working": "动作8",
    "waiting": "动作9",
    "error": "动作10",
    "result": "动作11",
    "success": "动作12"
  },

  // ── 外观 ──────────────────────────────
  "appearance": {
```

```
    "size": 160,                    // 画布长边，24–320
    "boxWidth": 160,                // 画布宽（只表达比例）
    "boxHeight": 160,               // 画布高
    "bgOpacity": 0,                 // 背景不透明度，0 = 全透明
    "avatar": "none",               // 不要内置头像
    "showDot": false,               // 不显示状态绿点
    "opacity": 1.0,                 // 整体不透明度
    "flipOnTurn": true,             // 转身时镜像
    "throwEnabled": true,           // 可甩出去弹墙
    "edgeSnap": false               // 是否吸附到边缘
  },

  // ── 自主走动 ──────────────────────────
  "move": {
    "enabled": true,
    "intervalSec": 30,
    "minDp": 40,
    "maxDp": 120
  },
  "moveParams": {
    "动作名": { "minDp": 50, "maxDp": 130 }
  },

  // ── 状态气泡 ──────────────────────────
  "statusAutoHideMs": 4000,
  "workStatusTexts": [
    ["idle 台词1", "idle 台词2"],
    ["thinking 台词"],
    ["working 台词"],
    ["waiting 台词"],
    ["error 台词"],
    ["result 台词"],
    ["success 台词"]
  ]
}
```

A.2 检查清单

发布前逐条过一遍：

- [ ] 所有 webm 都有 `TAG:ALPHA_MODE=1`
- [ ] 每个 webm ≤ 6 MB，合计 ≤ 12 MB
- [ ] 动作名只有 `a-z 0-9 - _`，≤ 24 字符
- [ ] `pools` 和 `actionMap` 里引用的动作名都在 `actions` 里存在

- [ ] `bgOpacity` 是 `0`
- [ ] `avatar` 是 `"none"`
- [ ] 所有动作裁成同一区域
- [ ] 至少配了 `idle`

附录 B ffmpeg 命令速查

```
# <a id="检查透明通道"></a>检查透明通道
ffprobe -v error -select_streams v:0 \
  -show_entries stream_tags=alpha_mode -of default=nw=1 x.webm

# <a id="png-序列-webm"></a>PNG 序列 → webm
ffmpeg -framerate 24 -i frame_%d.png -c:v libvpx-vp9 -pix_fmt yuva420p \
  -crf 30 -b:v 0 -metadata:s:v:0 alpha_mode="1" out.webm

# <a id="gif-webm"></a>GIF → webm
ffmpeg -i x.gif -c:v libvpx-vp9 -pix_fmt yuva420p \
  -crf 30 -b:v 0 -metadata:s:v:0 alpha_mode="1" out.webm

# <a id="裁剪空白"></a>裁剪空白
ffmpeg -c:v libvpx-vp9 -i x.webm -vf "crop=宽:高:X:Y" \
  -c:v libvpx-vp9 -pix_fmt yuva420p -crf 30 -b:v 0 \
  -metadata:s:v:0 alpha_mode="1" out.webm

# <a id="压缩"></a>压缩
ffmpeg -i x.webm -vf "scale=320:-2" -r 12 \
  -c:v libvpx-vp9 -pix_fmt yuva420p -crf 34 -b:v 0 \
  -metadata:s:v:0 alpha_mode="1" out.webm

# <a id="抽一帧看效果"></a>抽一帧看效果
ffmpeg -c:v libvpx-vp9 -i x.webm -vframes 1 frame.png

# <a id="查看视频信息"></a>查看视频信息
ffprobe -v error -show_streams x.webm
```

附录 C 参考实现

`dsh-app-pet` 是一个完整的参考实现（24 段动画、5 个动作池、七状态映射），可以直接看它的 `presets/pet.json` 学结构：

```
npm view dsh-app-pet
```

装到手机上体验：

```
dsh plugin --profile web add dsh-app-pet
```

文档基于 dsh-app-pet@1.0.3 编写。工作台在 Chrome 156 / ffmpeg 6.1 验证通过。